

Ideation Phase

Brainstorm & Idea Prioritization Template

Date	28 September 2026
Team ID	SWTID-2026-2056
Project Name	Incident Client Scripts and UI Policies
Maximum Marks	4 Marks

Brainstorm & Idea Prioritization Template:

This document presents a brainstorming and prioritization plan for improving the ServiceNow Incident form using Client Scripts and UI Policies. The goal is to guide users, improve data quality, and reduce incomplete or incorrect incident submissions.

Step-1: Team Gathering, Collaboration and Select the Problem Statement

Team goal: Improve the ServiceNow Incident form so users enter complete and correct information, while unnecessary fields remain hidden or read-only based on incident details.

Problem statement: How might we improve the ServiceNow Incident form so that users enter complete and correct information, while unnecessary fields remain hidden or read-only based on the incident details?

Current problems:

- Incidents may be submitted without important information.
- Irrelevant fields are displayed for every incident.
- Resolution information may be entered before an incident is resolved.
- Users may enter incomplete descriptions.
- Important fields are not always mandatory.
- Users may accidentally modify fields that should be read-only.

Brainstorming rules: Encourage every team member to share ideas, avoid criticizing ideas during brainstorming, focus on generating many possible solutions, combine similar ideas, and select solutions based on importance and feasibility.

Step-2: Brainstorm, Idea Listing and Grouping

Group	Ideas
Form usability	Hide irrelevant fields by category; display helpful messages; set Assignment group automatically; pop
Data validation	Make Short description and Description mandatory; warn when Description is too short; require Caller
Field control	Make Resolution notes and Resolution code mandatory when resolved; make closed records read-on
Automation	Calculate Priority from Impact and Urgency; set State when assigned; clear resolution fields when reo
User guidance	Show information messages, resolution warnings, troubleshooting guidance, and examples in the Des

Step-3: Idea Prioritization

Idea	Importance	Feasibility	Priority
Make Description mandatory	High	High	1
Require Resolution code and notes	High	High	2
Hide resolution fields until resolution	High	High	3

Idea	Importance	Feasibility	Priority
Make Configuration item mandatory by category	High	High	4
Automatically calculate Priority	High	Medium	5
Make closed incidents read-only	High	Medium	6
Display high-impact warning	Medium	High	7
Automatically set Assignment group	High	Medium	8

Final Selected Concept

A **guided Incident form** that uses UI Policies for field visibility, mandatory status, and read-only control, together with Client Scripts for validation, warnings, and automatic field calculations.

Expected benefits:

- Better-quality incident information.
- Fewer incomplete submissions.
- Faster assignment and resolution.
- Easier form navigation.
- Consistent incident-handling process.
- Reduced manual checking by support teams.

Implementation Plan

Recommended Client Scripts and UI Policies

1. Client Script: Description Validation

Type: onSubmit **Table:** Incident

Prevent submission when the Description is empty or contains fewer than 20 characters.

```
function onSubmit() {
  var description = g_form.getValue('description').trim();

  if (description.length < 20) {
    g_form.addErrorMessage('Please enter at least 20 characters in the Description field.');
```

```
    return false;
  }

  return true;
}
```

2. Client Script: Resolution Validation

Type: onSubmit **Table:** Incident

Require Resolution code and Resolution notes when the incident is resolved or closed.

```
function onSubmit() {
  var state = g_form.getValue('state');

  if (state == '6' || state == '7') {
    if (!g_form.getValue('close_code')) {
      g_form.addErrorMessage('Please select a Resolution code.');
```

```
    }
    if (!g_form.getValue('close_notes')) {
      g_form.addErrorMessage('Please enter Resolution notes.');
```

```
    }
  }
  return true;
}
```

Note: State values can differ in different ServiceNow instances. Confirm the values before implementation.

3. Client Script: Priority Calculation

Type: onChange **Field:** Impact or Urgency **Table:** Incident

```
function onChange(control, oldValue, newValue, isLoading) {
  if (isLoading) return;

  var impact = g_form.getValue('impact');
  var urgency = g_form.getValue('urgency');

  if (impact && urgency) {
    var matrix = [
      ['1-1', '1', '1-2', '2', '1-3', '3'],
      ['2-1', '2', '2-2', '3', '2-3', '4'],
      ['3-1', '3', '3-2', '4', '3-3', '5']
    ];
    g_form.setValue('priority', matrix[impact + '-' + urgency] || '');
  }
}
```

4. UI Policy: Resolved Incident Information

Field	Mandatory	Visible	Read-only
Resolution code	True	True	False
Resolution notes	True	True	False
Description	False	True	True
Short description	False	True	True

Condition: State is Resolved.

5. UI Policy: Category-based Configuration Item

Condition	Field	Mandatory	Visible	Read-only
Category is Hardware or Software	Configuration item	True	True	False

6. UI Policy: Hide Resolution Fields

Condition: State is not Resolved.

Hide Resolution code and Resolution notes, and make them non-mandatory until the incident is resolved.

7. Client Script: High-impact Warning

```
function onChange(control, oldValue, newValue, isLoading) { if (isLoading) return;

if (newValue == '1') {
    g_form.addInfoMessage('High-impact incident selected. Provide detailed information and verify the assignment group.');
```

Testing Checklist

- Test a new Incident form.
- Test all Impact and Urgency combinations.
- Confirm Description validation.
- Confirm Resolution code and notes become mandatory.
- Confirm resolution fields are hidden for open incidents.
- Test Hardware and Software categories.
- Confirm UI Policy reverse actions work when conditions change.
- Test with different user roles.
- Confirm client-side logic does not replace ACLs or Data Policies.

Implementation Recommendation

Use UI Policies first for simple field behavior such as mandatory, visible, and read-only settings. Use Client Scripts when scripting is required for validation, warnings, calculations, or complex conditions. ServiceNow documentation describes Client Scripts as browser-side scripts triggered by form events and UI Policies as a way to dynamically control form behavior. [web:2][web:4]